

EXP NO: 1	Write 8051 Assembly Language experiments using simulator

AIM:

To write an 8051 Assembly Language program to move a hexadecimal integer to the Accumulator and validate the results in the register window using the Keil simulator.

SOFTWARE REQUIRED:

S.No	Software Requirements	Quantity
1	Keil μ vision5 IDE	1

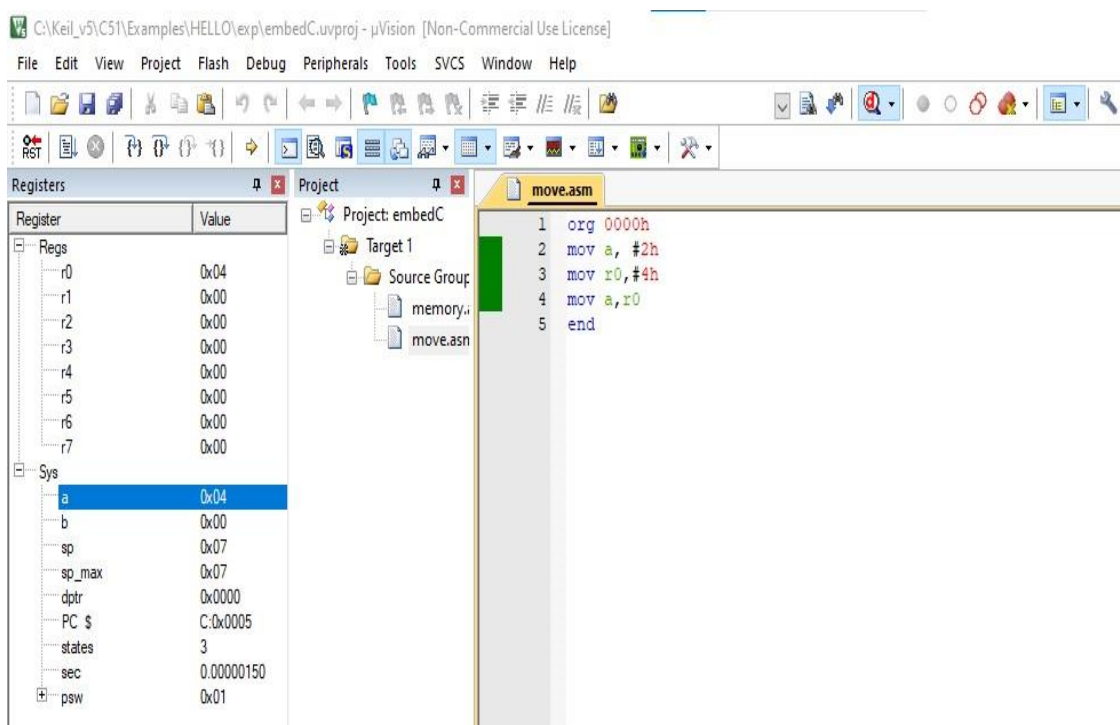
PROCEDURE:

1. Create new u vision project.
2. Select the folder (newly created) to save the project.
3. Save the project.
4. Select the vendor "Microchip" and device "AT89C51".
5. Addition of STARTUP.A51 to project folder.
6. STARTUP.A51 is added.
7. Type the Required Program.
8. Save the Program as Prgm.asm format.
9. Prgm.asm is to be added to Source Group1.
10. Select the file now it is added to Source Group1.
11. Build the target and then debug the target.
12. New window evaluation mode appeared. Click ok
13. Verify the program output by clicking step option or F11. Output can be viewed in the register window

PROGRAM:

```
ORG 0000H
MOV A, #02H
MOV R0, #04H
MOV A, R0
END
```

OUTPUT SCREENSHOT:



VIVA- VOCE QUESTIONS:

1. What is microcontroller?
2. What is the use of assembler directives ORG, END?
3. Explain MOV A, R0 instruction.
4. Define assembler.
5. Why filename is saved as .asm extension?

RESULT:

EXP NO : 2	Test data transfer between registers and memory
DATE:	

AIM:

To write an assembly language program to test the data transfer between registers and memory using Keil Simulator.

SOFTWARE REQUIRED:

S.No	Software Requirements	Quantity
1	Keil μ vision5 IDE	1

PROCEDURE:

1. Create new μ vision project.
2. Select the folder (newly created) to save the project.
3. Save the project.
4. Select the vendor "Microchip" and device "AT89C51".
5. Addition of STARTUP.A51 to project folder.
6. STARTUP.A51 is added.
7. Type the Required Program.
8. Save the Program as Prgm.asm format.
9. Prgm.asm is to be added to Source Group1.
10. Select the file now it is added to Source Group1.
11. Build the target and then debug the target.
12. New window evaluation mode appeared. Click ok.
13. Modify the memory address by right clicking at the particular hexadecimal number and give the appropriate data to the memory address.
14. Verify the program output by clicking step option or F11. Output can be viewed in the register window and in memory window at the bottom.

PROGRAM:

```
ORG 0000H
MOV R0, #40H
MOV R1, #50H
MOV R3, #03H
BACK: MOV A, @R0
      MOV B, @R1
      MOV @R0, B
      MOV @R1, A
      INC R0
      INC R1
      DJNZ R3, BACK
      END
```

The screenshot shows the uVision IDE with the following components:

- Menu Bar:** File, Edit, View, Project, Flash, Debug, Peripherals, Tools, SVCS, Window, Help.
- Toolbox:** Contains various icons for project management, debugging, and assembly.
- Registers Window:**
 - Reg:** r0 (0x40), r1 (0x50), r2 (0x00), r3 (0x03), r4 (0x00), r5 (0x00), r6 (0x00), r7 (0x00).
 - Sys:** a (0x01), **b (0x06)**, sp (0x07), sp_max (0x07), dptr (0x0000), PC \$ (C:0x0009), states (6), sec (0.00000300), psw (0x01).
- Project Window:**
 - Project: embedC
 - Target 1
 - Source Group 1
 - datatransfer.
- Assembly Code Window (datatransfer.asm):**

```

1  ORG 0000H
2  MOV R0, #40H
3  MOV R1, #50H
4  MOV R3, #03H
5  BACK: MOV A, @R0
6  MOV B, @R1
7  MOV @R0, B
8  MOV @R1, A
9  INC R0
10 INC R1
11 DJNZ R3, BACK
12 END

```
- Memory Window:**
 - Address: 0x40H
 - Content: I:0x40: 01 03 05 00 00 00 00 00 00 00 00 00 00 00 00 00 06 08 09 00 00 00 00 00 00 00

The screenshot shows the Keil uVision IDE interface. The main window displays the assembly code for 'datatransfer.asm'. The code is as follows:

```

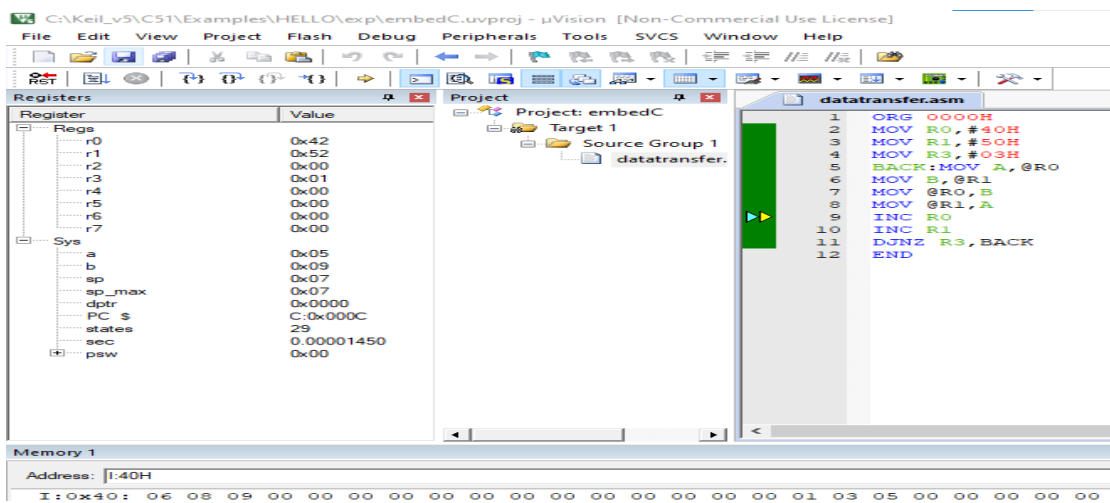
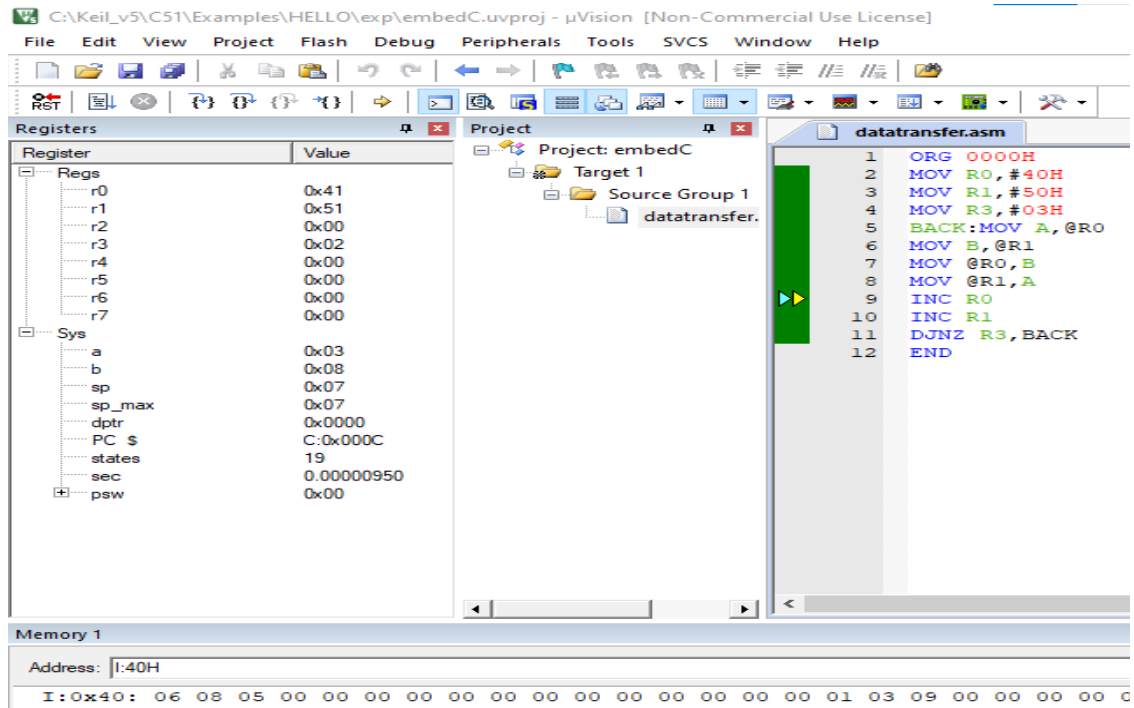
1  ORG 0000H
2  MOV R0, #40H
3  MOV R1, #50H
4  MOV R3, #03H
5  BACK: MOV A, @R0
6  MOV B, @R1
7  MOV @R0, B
8  MOV @R1, A
9  INC R0
10 INC R1
11 DJNZ R3, BACK
12 END

```

The Project window shows the file structure:

- Project: embedC
 - Target 1
 - Source Group 1
 - datatransfer.

The Memory window shows the address 0x40.



VIVA- VOCE QUESTIONS:

1. Explain DJNZ R3, BACK instruction.
2. What is Register addressing mode?
3. Define addressing modes of 8051.

RESULT:

EXP NO: 3	Perform ALU operations.
DATE:	

AIM:

To write an 8051 Assembly Language program and Perform ALU operations (Addition, subtraction, Division and Multiplication, AND, OR, XOR) using Keil simulator.

SOFTWARE TOOLS REQUIRED:

S.No	Software Requirements	Quantity
1	Keil μ vision5 IDE	1

PROCEDURE:

1. Create new u vision project.
2. Select the folder (newly created) to save the project.
3. Save the project.
4. Select the vendor "Microchip" and device "AT89C51".
5. Addition of STARTUP.A51 to project folder.
6. STARTUP.A51 is added.
7. Type the Required Program.
8. Save the Program as Prgm.asm format.
9. Prgm.asm is to be added to Source Group1.
10. Select the file now it is added to Source Group1.
11. Build the target and then debug the target.
12. New window evaluation mode appeared. Click ok
13. Verify the program output by clicking step option or F11. Output can be viewed in the register window and in memory window at the bottom (only for multiplication and division operations).

PROGRAM:

ORG 0000H

CLR C

//ADDITION

MOV A, #20H

ADD A, #21H

MOV 41H, A

//SUBTRACTION

MOV A, #20H

SUBB A, #18H

MOV 42H, A

//MULTIPLICATION

MOV A, #03H

MOV B, #04H

MUL AB

MOV 43H, A

//DIVISION

MOV A, #95H

MOV B, #10H

DIV AB

MOV 44H, A

MOV 45H, B

//AND

MOV A, #25H

MOV B, #12H

ANL A, B

MOV 46H, A

//OR

MOV A, #25H

MOV B, #15H

ORL A, B

MOV 47H, A

//XOR

MOV A, #45H

MOV B, #67H

XRL A, B

MOV 48H, A

```
//NOT
MOV A, #45H
CPL A
MOV 49H, A
END
```

OUTPUT SCREENSHOT OF ALU OPERATION

The screenshot displays the uVision IDE interface for a project named "E:\emb\ex3.uvproj". The main window shows the assembly code for "ex3.asm". The code includes instructions for clearing the carry flag, moving values into registers A and B, performing subtraction, multiplication, and division. A "Memory 1" window is open, showing a memory dump starting at address D:40H. The dump displays hexadecimal values for various memory locations, including D:0x40, D:0x59, D:0x72, D:0x8B, D:0xA4, D:0xBD, and D:0xD6. The registers window on the left shows the current state of the registers, with R0 through R7 all containing 0x00. The command window at the bottom shows the command "Load 'E:\\emb\\Objects\\ex3'" and an error message: "*** error 65: access violation at C:0x0040 : no 'execute/read' permission".

Registers

Register	Value
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
a	0xba
b	0x67
sp	0x07
sp_max	0x07
PC	\$
auxr1	0x00
dptr	0x00
states	43
sec	0.00
paw	0x01

Disassembly

```

C:0x0040 00 NOP
C:0x0041 00 NOP
C:0x0042 00 NOP
C:0x0043 00 NOP

```

ex3.asm

```

1 ORG 0000H
2 CLR C
3 MOV A, #20H
4 ADD A, #21H
5 MOV 41H, A
6
7 // SUB
8 MOV A, #06H
9 SUBB A, #04H
10 MOV 42H, A
11
12 // MUL
13 MOV A, #03H
14 MOV B, #04H
15 MUL AB
16 MOV 43H, A
17
18 //DIV
19 MOV A, #09H
20 MOV B, #03H
21 DIV AB

```

Memory 1

Address: D:40H

D:0x40:	00 41 02 0C 03 00 00 37 22 BA 00
D:0x59:	00 00
D:0x72:	00 00
D:0x8B:	00 00 00 08 00 FF 00
D:0xA4:	00 00
D:0xBD:	00 00 00 FF 00 00 14 00 00 FF 00
D:0xD6:	00 00

Command

```

Load "E:\\emb\\Objects\\ex3"
*** error 65: access violation at C:0x0040 : no 'execute/read' permission

```

VIVA – VOCE QUESTIONS:

1. Explain arithmetic instructions.
2. Explain MUL AB instruction.
3. Explain DIV AB instruction.
4. Explain MOV 42H, A instruction.

RESULT:

EXP NO: 4	Write Basic and arithmetic Programs Using Embedded C
DATE:	

AIM:

To write a basic arithmetic program (for performing Addition, subtraction, Division and Multiplication) using Embedded C in Keil Simulator.

SOFTWARE REQUIRED:

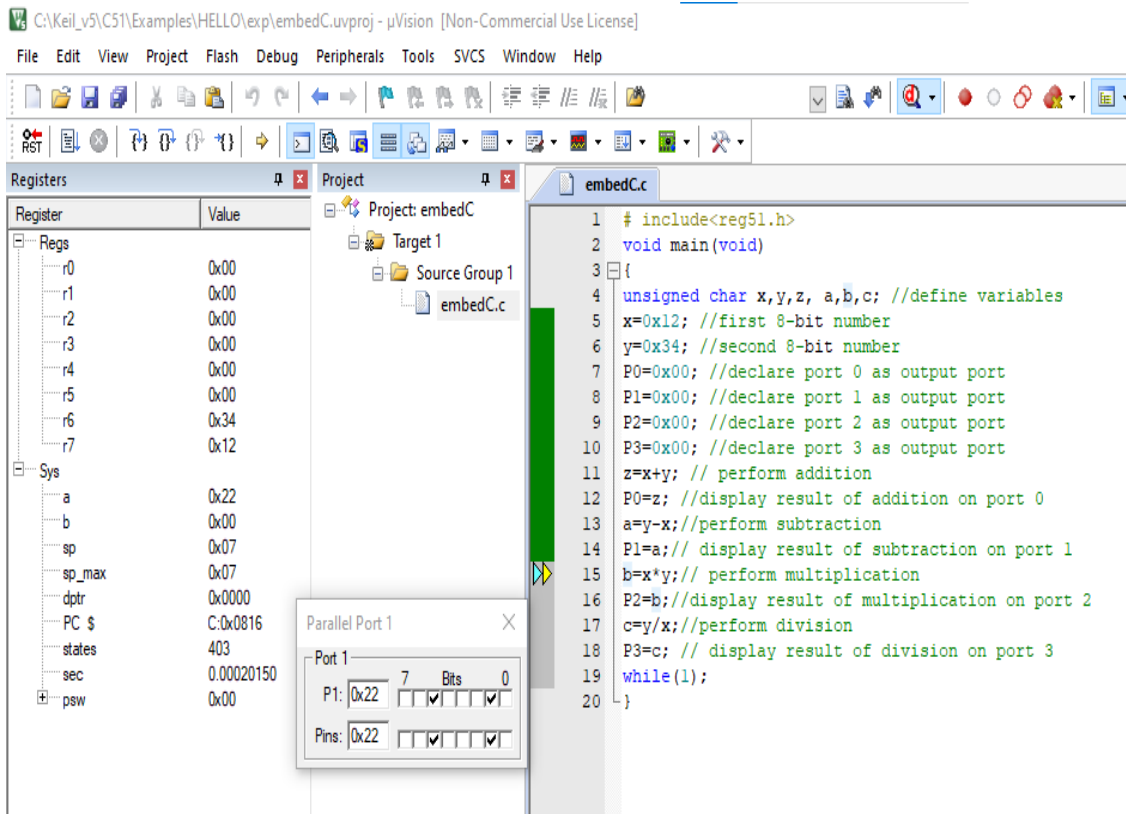
S.No	Software Requirements	Quantity
1	Keil μ vision5 IDE	1

PROCEDURE:

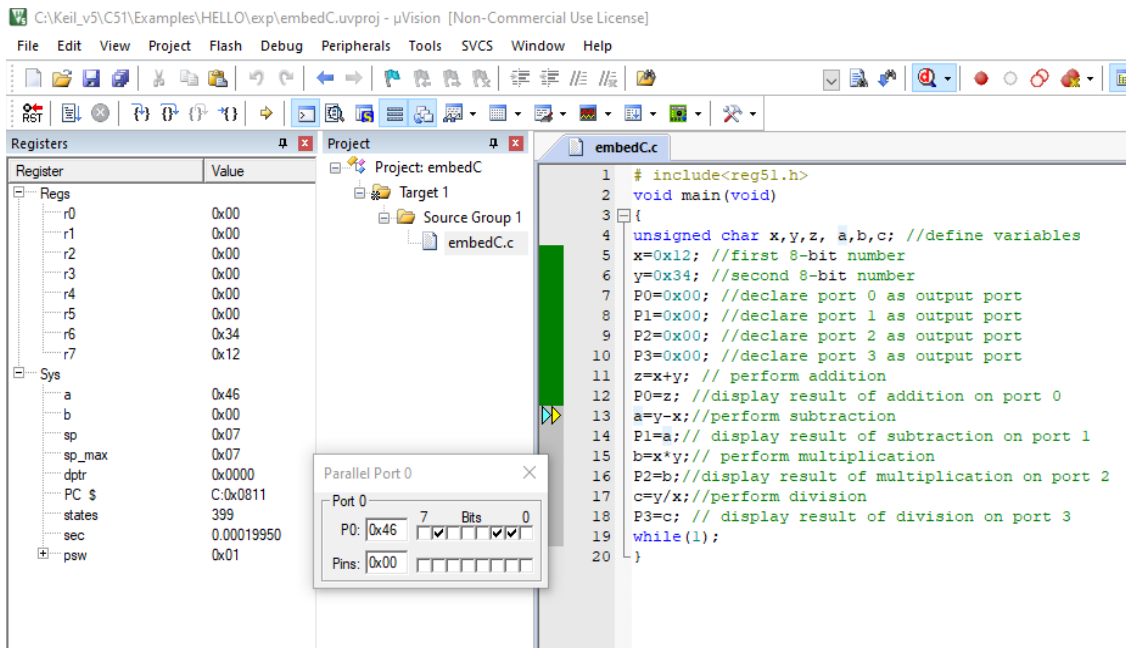
1. Create new u vision project.
2. Select the folder (newly created) to save the project.
3. Save the project.
4. Select the vendor "Microchip" and device "AT89C51".
5. Addition of STARTUP.A51 to project folder.
6. STARTUP.A51 is added.
7. Type the Required Program.
8. Save the Program as Prgm.C format.
9. Prgm.C is to be added to Source Group1.
10. Select the file now it is added to Source Group1.
11. Build the target and then debug the target.
12. New window evaluation mode appeared. Click ok
13. Verify the program output by clicking step option or F11. Output can be viewed in theregister window
14. In peripherals I/O Ports
 - ☐ Port 0 displays result Addition
 - ☐ Port 1 displays result Subtraction
 - ☐ Port 2 displays result of multiplication
 - ☐ Port 3 displays result of division

PROGRAM:

```
#include<reg51.h>
void main(void)
{
    unsigned char x,y,z,a,b,c;           //define variables
    x=0x12;                             //first 8-bit number
    y=0x34;                             //second 8-bit number
    P0=0x00;                             //declare port 0 as output port
    P1=0x00;                             //declare port 1 as output port
    P2=0x00;                             //declare port 2 as output port
    P3=0x00;                             //declare port 3 as output port
    z=x+y;                               // perform addition
    P0=z;                                //display result of addition on port 0
    a=y-x;                               //perform subtraction
    P1=a;                                // display result of subtraction on port 1
    b=x*y;                               // perform multiplication
    P2=b;                                //display result of multiplication on port 2
    c=y/x;                               //perform division
    P3=c;                                // display result of division on port 3
    while(1);
}
```

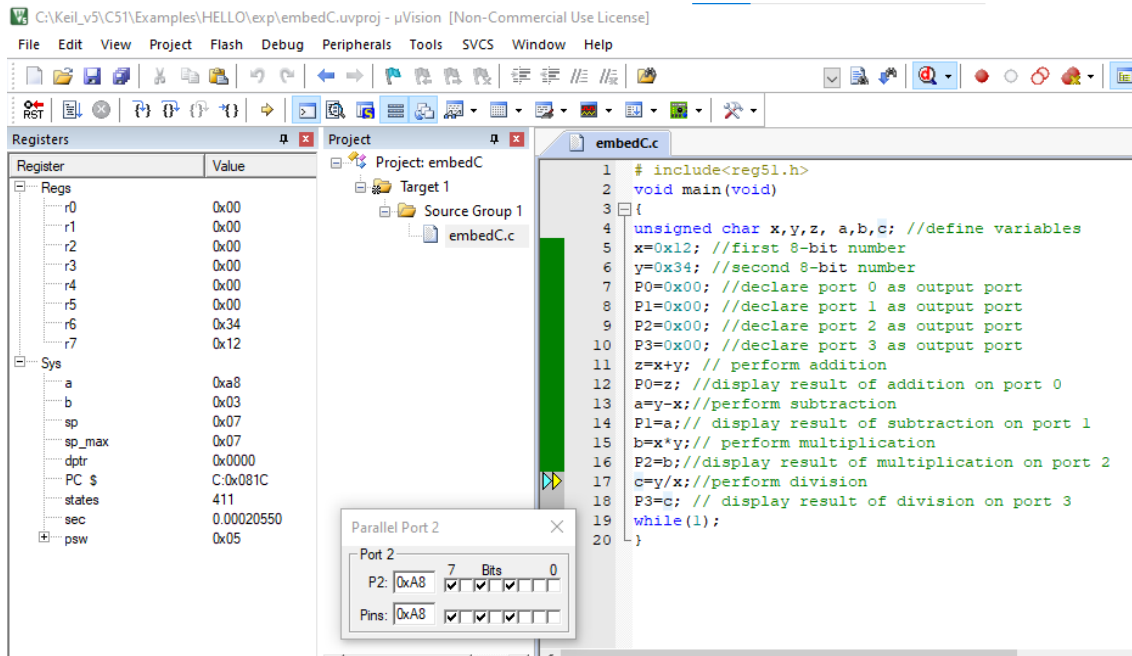



OUTPUT SCREENSHOT OF PARALLEL PORT 1 SUBTRACTION

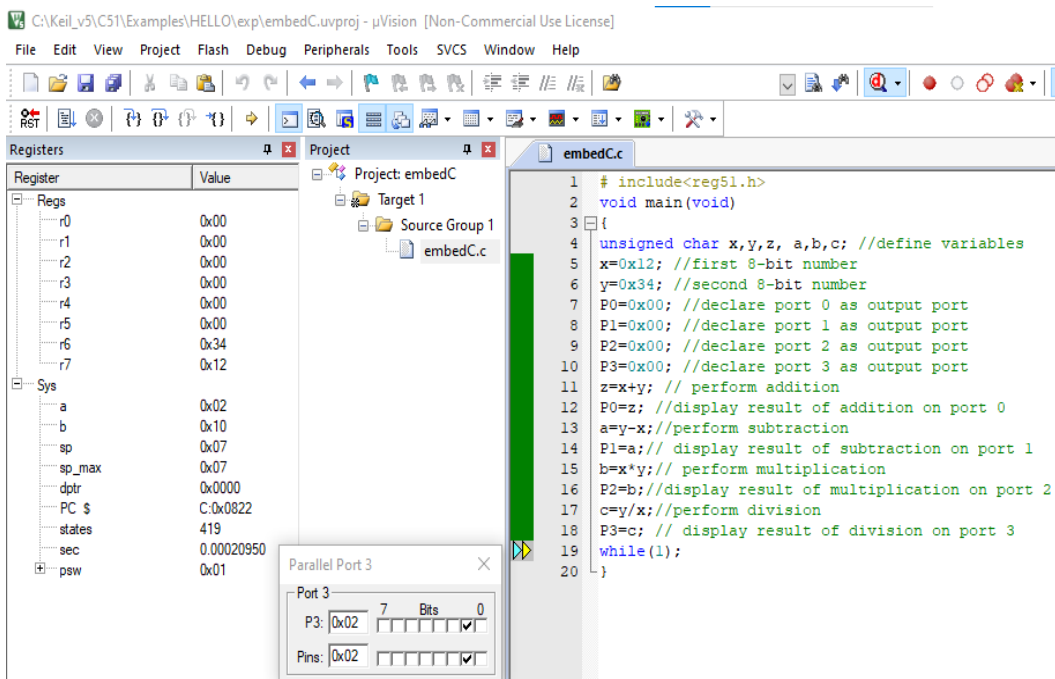


OUTPUT SCREENSHOT OF PARALLEL PORT 0 ADDITION

OUTPUT SCREENSHOT OF PARALLEL PORT 2 MULTIPLICATION



OUTPUT SCREENSHOT OF PARALLEL PORT 3 DIVISION



VIVA- VOCE QUESTIONS:

1. What is embedded C?
2. How many ports are available in 8051?
3. What is variables in embedded C?
4. What is the difference between C and Embedded C?

RESULT:

